02-620 Week 6

Machine Learning for Scientists

Aidan Jan

February 20, 2026

Support Vector Machines

Generative vs. Discriminative Classifiers

- Training classifiers involves estimating $f : X \to Y$ or $P(Y|X)$
- Generative classifiers (e.g., Naive Bayes)
  - Model and learn $P(Y, X) = P(X|Y)P(Y)$
  - Derive $P(Y|X)$ from $P(Y, X)$ using Bayes rule
- Discriminative classifiers (e.g., logistic regression)
  - Model $P(Y|X)$ directly
- SVM is
  - Another discriminative classifier
  - Non-probabilistic

Recap: Linear/Logistic Classifier

Recall our linear regression classifiers colassify $x = (x_1, x_2)$ into $y = 0$ or $y = 1$.

- Our objective function is
$$P(Y = 1|x; \theta) = \frac{1}{1 + e^{-\left(\theta_0 + \sum_{j=1}^{D} \theta_j x_j\right)}}$$
- The decision boundary is
$$\theta_0 + \sum_{j=1}^{D} \theta_j x_j$$
- Many different optimal classifiers (decision boundaries) are possible, as long as they divide the data correctly

Alternative criterion: **Max-margin principle**

- Instead of fitting all points, focus on boundary points
- Learn a boundary that leads to the **largest margin** from both sets of points

The margin is a set of **parallel lines** on each side of the decision boundary, that are parallel to the decision boundary, and separate the two classes by the widest margin possible.

- The sides of the boundaries are referred to as **support vectors**.
- The distance between the boundaries are represented by M, or the width of the margin.

Support Vector Machines: Model and Learning

Data

- N data points with D features: $x_i \in \mathbb{R}^D$
- Labels: $y_i \in \{-1, 1\}$

Model

- Linear model:

$$y_i = \begin{cases} 1 & w^T x + b > 0 \\ -1 & w^T x + b < 0 \end{cases}$$

- Parameters: $\theta = (b, w_1, \ldots, w_D)$

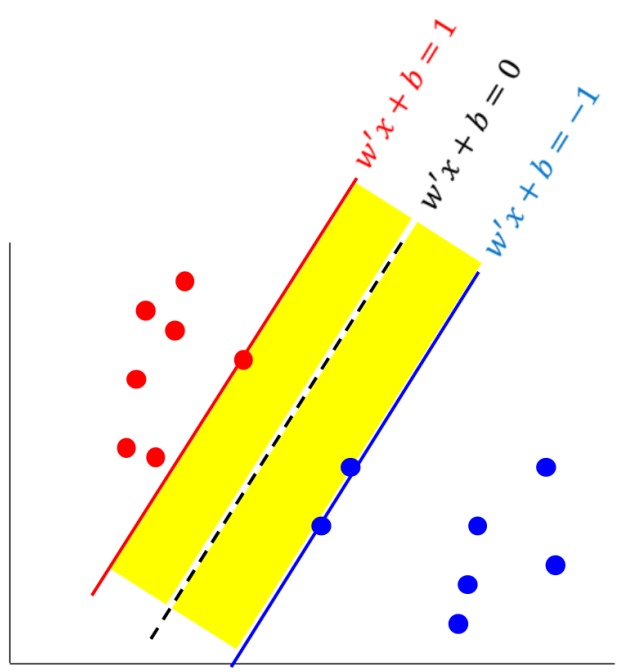
Learning

- Max margin principle: maximize the distance between the two lines (hyperplanes) subject to correct classification of all points.
- For the hard-margin version: $\arg\min_{w,b} \frac{w^T w}{2}$, such that $y_i(w^T x_i + b) \geq 1$ for all i
- For the soft-margin version: $\arg\min_{w,b} \frac{w^T w}{2} + C \sum_{i=1}^{N} \epsilon_i$ such that $y_i(w^T x_i + b) \geq 1 - \epsilon_i$, and $\epsilon_i \geq 0$ for all i

Support Vector Machine with Hard Margin

We can use the following classification:

- $y = 1$ if $w'x + b \geq 1$
- $y = 0$ if $w'x + b \leq -1$
- Undefined if $-1 \leq w'x + b \leq 1$



Let's define the width of the margin to be M. How can we encode our goal of maximizing M in terms of our parameters, (w and b) and training data?

- It turns out that maximizing the margin M is the same as minimizing the inverse of M.
- The distance between the two hyperplanes $w^T x + b = 1$ and $w^T x + b = -1$ is $\frac{2}{\sqrt{w^T w}}$. Therefore, we can minimize $\left(\frac{w^T w}{2}\right)$
- We still have the constraints:
  - For all x in class $+1$, $w'x + b \geq 1$
  - For all x in class -1, $w'x + b \leq -1$
- A total of N constraints if we have N input samples

SVM as Quadratic Programming (QP)

Our objective to minimize $\frac{w^T w}{2}$ is a quadratic function of w. The constraints $w'x + b \leq -1$ and $w'x + b \geq 1$ are linear.

- We can formulate this problem as a quadratic programming problem, and just plug it into a quadratic programming solver.

Note: A quadratic programming solver is similar to a linear programming solver, except a degree higher.

Support Vector Machine with Soft Margin

What if our data is *not linearly separable*?

- Most of the time, our data will have noise or outliers.
- Instead of just maximizing the margin, what if we minimize the training data error as well?
  - Instead of minimizing the number of misclassified points, we can minimize the *distance* between these points and their correct plane
- We can write the new optimization problem as

$$min_w \frac{w'w}{2} + \sum_{i=1}^{n} C\epsilon_i$$

  subject to the following constraints:
  - For all x in class $+1$:
  $$w'x + b \geq 1 - \epsilon_i$$
  - For all x in class -1:
  $$w'x + b \leq -1 + \epsilon_i$$
- We also want $\epsilon_i \geq 0$. We now have a total of $2N$ constraints.

Reframing the Problem

- Instead of solving these quadratic programming problems directly, we will solve a dual formulation of the SVM optimization problem.
- The main reason for switching to this type of representation is that it would allow us to identify **support vectors** and to use a neat trick that will make our lives easier (and the runtime faster!)

Dual Formulation of QP: A toy example

- In general, Lagrange multipliers can be applied to turn the original primal problem into a dual problem.
- Primal problem: $\min_x x^2$ such that $x \geq b$. The primal parameter is x.
- Lagrangian: $\max_\alpha \min_x x^2 + \alpha(b - x)$ such that $\alpha \geq 0$. α is a Lagrange multiplier associated with each constraint.

$$\begin{aligned} \frac{\mathrm{d}}{\mathrm{d}x} x^2 + \alpha(b - x) &= 0 \\ x &= \alpha/2 \\ \alpha^2/4 - \alpha(\alpha/2 - b) &= -\alpha^2/4 + \alpha b \end{aligned}$$

We can then write the Lagrangian as:

$$\max_\alpha -\alpha^2/4 + \alpha b$$

such that $\alpha \geq 0$. This is the dual problem. The dual parameter is α

Dual Formulation of QP: SVM with hard-margin

Primal formulation:

$$\min_w \frac{w^T w}{2}$$

such that $(w^T x_i + b) y_i \geq 1$, for all i.

This can be written as the Lagrangian:

$$\max_\alpha \min_{w,b} \frac{w^T w}{2} + \sum_{i=1}^N \alpha_i [1 - (w^T x_i + b) y_i]$$

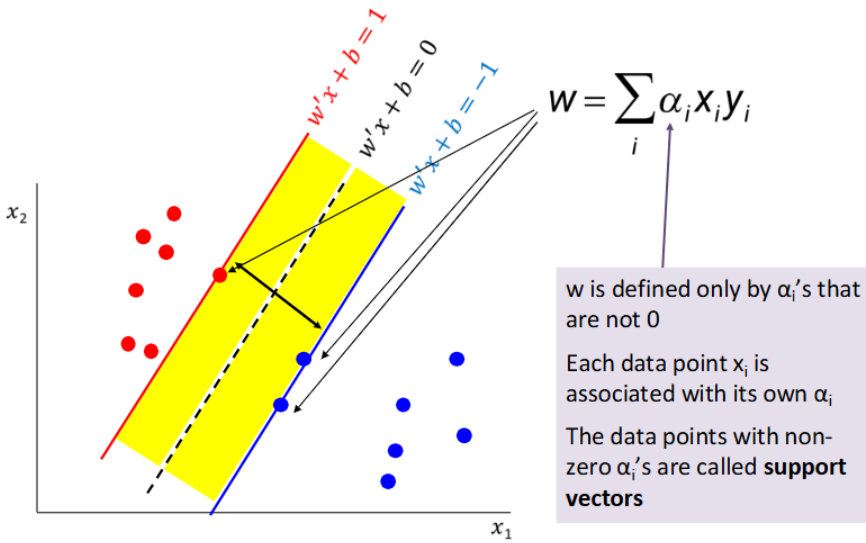
such that $\alpha_i \geq 0$, for all i.

Finally, converting to the dual formulation,

$$\max_{\alpha_i} \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j y_i y_j x_i^T x_j$$

where $\sum_{i=1}^N \alpha_i y_i = 0$, and $\alpha_i \geq 0$, for all i.

If we differentiate with respect to w, b, $w = \sum_i \alpha_i x_i y_i$, this tells us about support vectors.



$$w^T x_j + b = \sum_{i=1}^{N} \alpha_i y_i x_i^T x_j + b$$

To evaluate a new sample x_j, we need to compute: dot product with support vectors in the training data.

Dual Formulation of QP: SVM with soft-margin

Primal formulation:

$$\min_{w,b} \frac{w^T w}{2} + C \sum_{i=1}^{N} \epsilon_i$$

such that $(w^T x_i + b) y_i \geq 1 - \epsilon_i$, for all i. $\epsilon_i \geq 0$, for all i.

Dual formulation:

$$\max_{\alpha_i} \sum_{i=1}^{N} \alpha_i - \frac{1}{2} \sum_{i=1}^{N} \sum_{j=1}^{N} \alpha_i \alpha_j y_i y_j x_i^T x_j$$

$\sum_{i=1}^{N} \alpha_i y_i = 0$, and $C \geq \alpha_i \geq 0$, for all i. The only difference is that the α_i's are now bounded.

Summary

Support vector machine:

- Instead of fitting all points, focus on boundary points
- Learn a boundary that leads to the largest margin from both sets of points

SVM Primal vs. Dual formulations:

- Primal formulation:

$$\min_{w} \frac{w^T w}{2} \quad \text{s.t. } (w^T x_i + b) y_i \geq 1, \forall i$$

- Dual formulation:

$$\max_{\alpha_i} \sum_{i=1}^{N} \alpha_i - \frac{1}{2} \sum_{i=1}^{N} \sum_{j=1}^{N} \alpha_i \alpha_j y_i y_j x_i^T x_j, \quad \text{s.t. } \sum_{i=1}^{N} \alpha_i y_i = 0, \alpha_i \geq 0, \forall i$$

	Primal Formulation	**Dual Formulation**
Parameters	w, b	α_i
Number of Parameters	# features	# training samples, N
What parameters mean	Decision boundary margin	Support vectors α_i
How are parameters related?		$w = \sum_i \alpha_i x_i y_i$

Computational Cost for Testing

- During testing, the computational costs using primal vs. dual representations are:
  - Primal: $y_{new} = \text{sign}(w^T x_{new} + b)$
  - Dual: $y_{new} = \text{sign}\left(\sum_{i=1}^{n} \alpha_i y_i x_i' x_{new} + b\right)$
- If one uses dual parameters to make predictions, the prediction depends only on the support vectors, but this is not explicitly represented in the primal.

Runtime notes:

- $w^T x_{new}$ takes D operations, for D features.
- $x_i' x_{new}$ takes Dr operations, where r is the number of support vectors, ($\alpha_i > 0$). This is a dot product with all training samples.

Dual SVM - Interpretation for linearly non-separable case

In the equation

$$w = \sum_i \alpha_i x_i y_i$$

Points on the wrong side of the margin (or between the margins) would have α_i's that are not zero.

Non-linear SVM: Kernel Trick

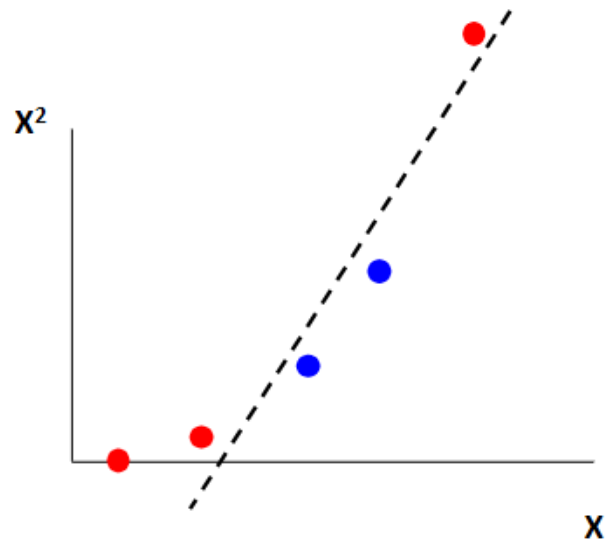
Consider a 1D number line. Positions are labeled with classes.

```
--------(1)-----(1)--------------(2)---------(2)--------------
```

A SVM can correctly classify this data by having the decision boundary down the middle. But what about the next one?

```
--------(1)-----(1)--------------(2)---------(2)--------(1)---
```

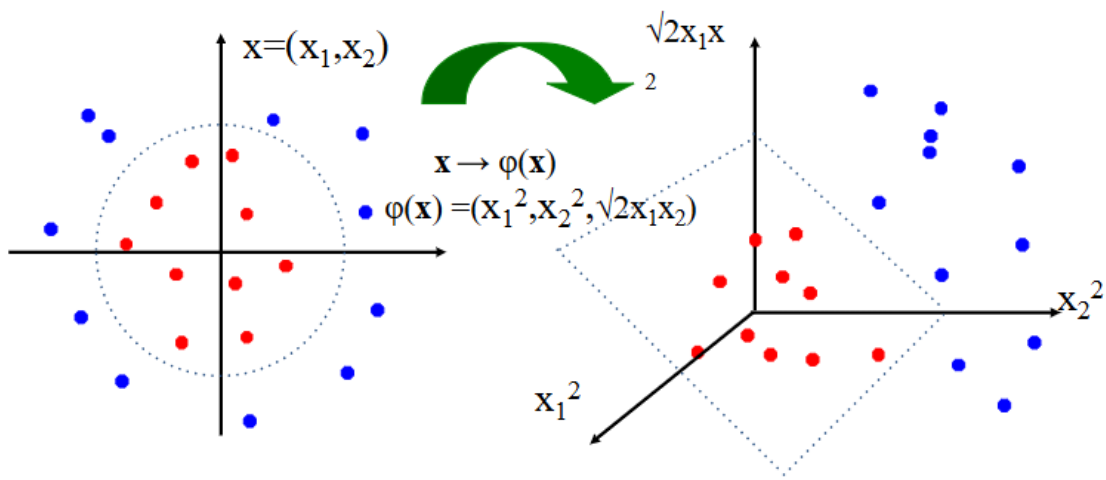
Normally, an SVM cannot classify this data. However, what if we mapped this number line onto a 2D plane, following the curve x^2?



By transforming the input space from x to a higher dimensional space $\phi(x) = (x, x^2)$, the training set is now linearly separable.

Classifying Based on 2D Input

The original input space x can be mapped to some higher dimensional feature space $\phi(x)$ where the training set is separable:



- Original problem: $\max_{\alpha_i} \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j y_i y_j x_i^T x_j$ such that $\sum_{i=1}^N \alpha_i y_i = 0$, and $\alpha_i \geq 0$, for all i.
- After input transformation: $\max_{\alpha_i} \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j y_i y_j \phi(x_i)^T \phi(x_j)$ such that $\sum_{i=1}^N \alpha_i y_i = 0$, and $\alpha_i \geq 0$, for all i.
  - Red part is the kernek, κ

Transformation of Inputs

- Possible problems
  - High computation burden due to high-dimensionality

  - Many more parameters

- SVM solves these two issues simultaneously
  - "Kernel tricks" for efficient computation
  - Dual formulation only assigns parameters to samples, not features

Kernel Trick

- Consider feature $x = [x_1, x_2]$
- Map it to a 6-dimensional space

$$\phi(x) = [1, \sqrt{2}x_1, \sqrt{2}x_2, x_1^2, x_2^2, \sqrt{2}x_1x_2]^T$$

- Compute $\phi(x)^T\phi(x')$: what's the time complexity?
- Kernel:

$$\phi(x)^T\phi(x') = \kappa(x, x') = (x^Tx' + 1)^2$$

Why do SVMs work?

- If we are using huge features spaces (via kernels), why don't we overfit the data?
  - Number of parameters remains the same (and most are set to 0)
  - While we have a lot of input values, at the end we only care about the support vectors and these are usually a small group of samples
  - The minimization (or the maximizing of the margin) function acts as a sort of regularization term leading to reduced overfitting

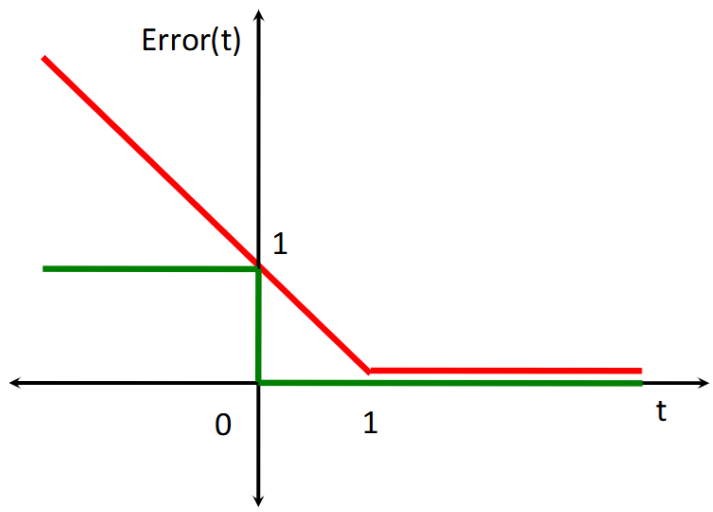
Multi-class Classification with SVMs

- What if we have data from more than two classes?
- Most common solution: One vs. all (Create a classifier for each class against all other data)

Separable vs. Non-separable cases

- In separable cases, we minimize $\frac{w^Tw}{2}$ subject to the constraints $(w'x_i + b)y_i \geq 1$.
- For the non-separable case, we minimize $\frac{w^Tw}{2} + C\sum_{i=1}^{N}\epsilon_i$ subject to the constraints $(w'x_i + b)y_i \geq 1 - \epsilon_i$

Error Function for SVM



- Let $t = (w'x_i + b)y_i$. $t > 0$ for both positive and negative training samples if classified correctly.
- Ideal classifier:

$$\text{Error}(t) = \begin{cases} 0 & t > 0 \\ 1 & t < 0 \end{cases}$$

- SVM: $\text{Error}(t) = [1 - t]_+$. The $[]_+$ part denotes the positive part. (Hinge Loss)